



AULA 3

Banco de dados: complemento SQL DML, consultas e visões relacionais, índices e projeto físico



Objetivos de aprendizagem da aula

Ao final desta aula, você irá:

- Expressar consultas em SQL com duas ou mais tabelas.
- Conhecer Operadores de conjuntos e junções relacionais.
- Aplicar funções e agregados.
- Utilizar o conceito de visões relacionais.
- Utilizar Projeto físico e análise de índices.



Que história é essa?

Esta disciplina tem por objetivo ensinar ao aluno os fundamentos principais de bancos de dados, com equilíbrio na apresentação entre teoria e prática, nessa área tão relevante da computação e ciência de dados em geral.

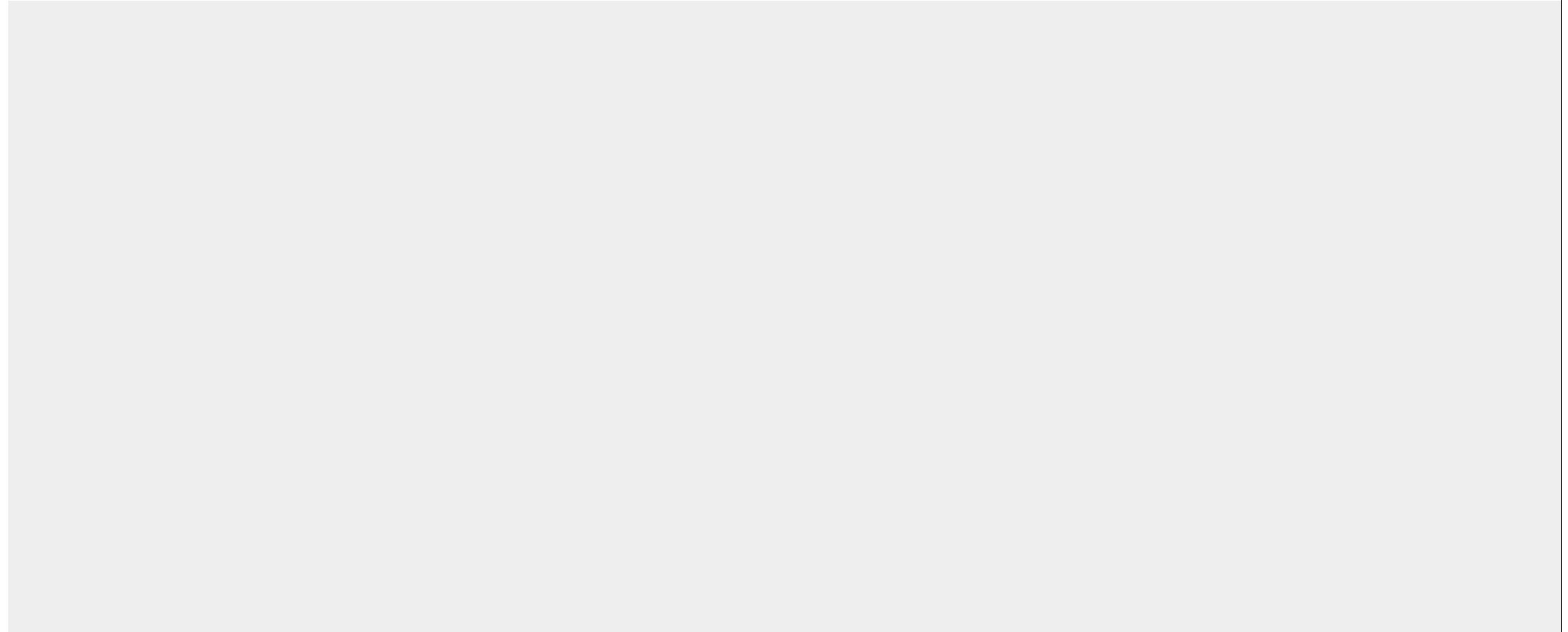
Mantendo o foco!

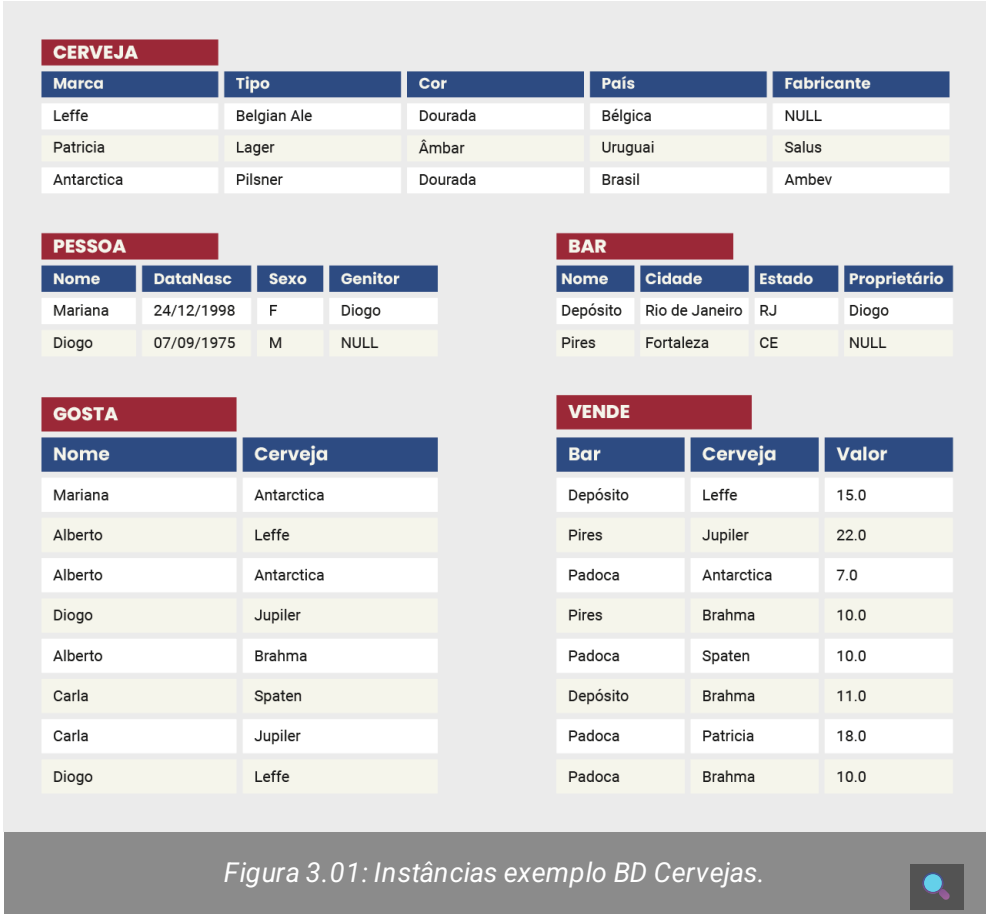
Nas aulas anteriores, foram discutidos os conceitos fundamentais do modelo relacional, os passos principais de um projeto de banco de dados conceitual e lógico e a linguagem SQL, tanto DML (para consultas e atualizações) como DDL, para criação de objetos. Nesta terceira aula, vamos ver alguns aspectos mais avançados dos sistemas de bancos de dados relacionais.

Inicialmente, retomamos a linguagem SQL em sua fração DML de consultas, agora envolvendo mais de uma tabela na expressão lógica da consulta. Em particular, daremos destaque ao operador de junção, um dos mais importantes para a linguagem na prática. Veremos também dois conceitos adicionais: as visões e os índices. Ambos são relevantes para o projeto físico de bancos de dados relacionais, a última etapa de projeto, já voltada para a melhoria de desempenho.

Até o momento, nós vimos como utilizar a linguagem SQL para consultas relativamente simples. É possível selecionar colunas que desejamos visualizar (cláusula SELECT) e determinar qual tabela será necessária (cláusula FROM). Podemos também ordenar o resultado na saída através da cláusula ORDER BY. Por fim, pode ser necessário filtrar dados e elementos que satisfaçam condições particulares. Para isso, temos a cláusula WHERE, que utiliza predicados lógicos (expressões cujo resultado é verdadeiro ou falso) para restringir quais tuplas devem constar na resposta.

Veremos, em seguida, outros exemplos de manuseios possíveis e interessantes com bancos de dados relacionais. Vamos considerar o mesmo esquema relacional das cervejas, projetado e construído na aula passada, ilustrado na seguinte figura:

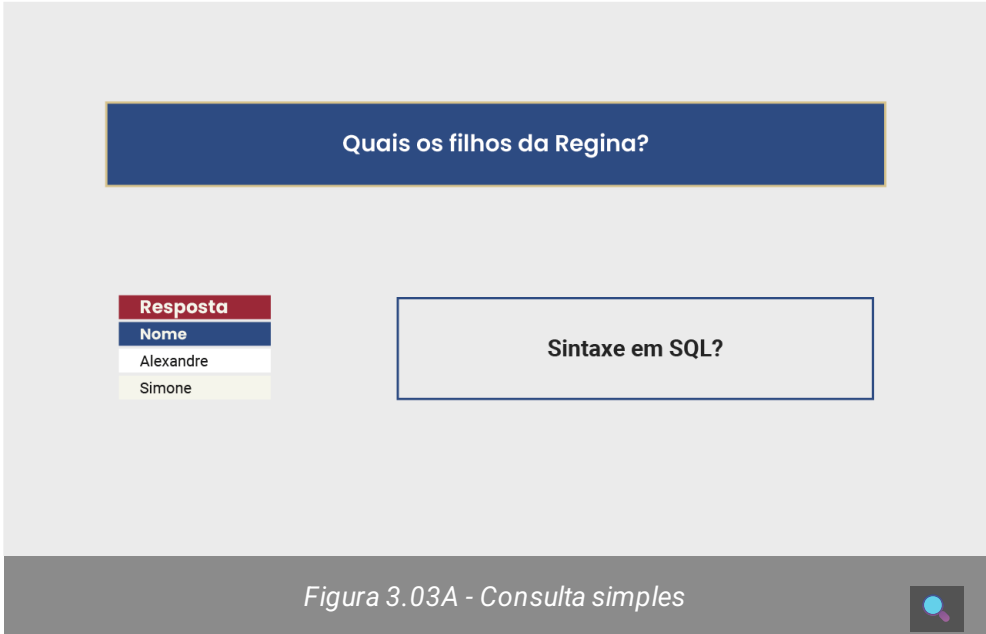




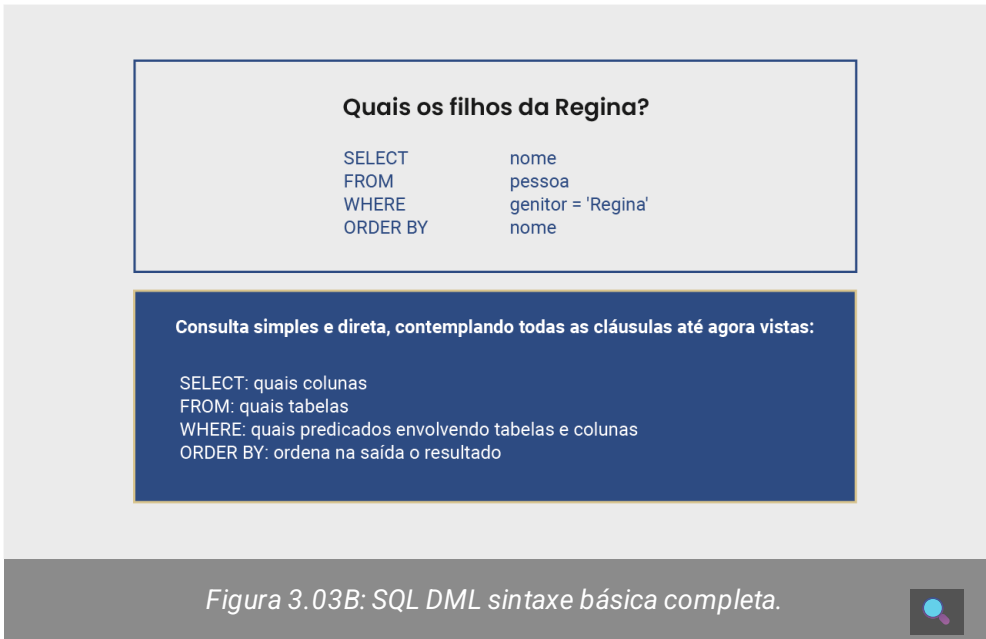
Seja, também, uma instância particular da tabela Pessoa, na figura a seguir, que utilizaremos ao longo desta aula para conhecer novas sintaxes e operadores. Cabe observar que são 12 tuplas com valores fictícios, utilizados apenas para fins didáticos e de exemplo. Por exemplo, temos a informação do genitor (Diogo) da Mariana, mas não sabemos quem é o genitor do Diogo, por isso consta como NULL. Lembrar que Genitor é FK na tabela Pessoa, referenciando a coluna Nome, PK na mesma tabela. Para PKs não podemos ter nulos, mas, para FKs, isso é permitido.



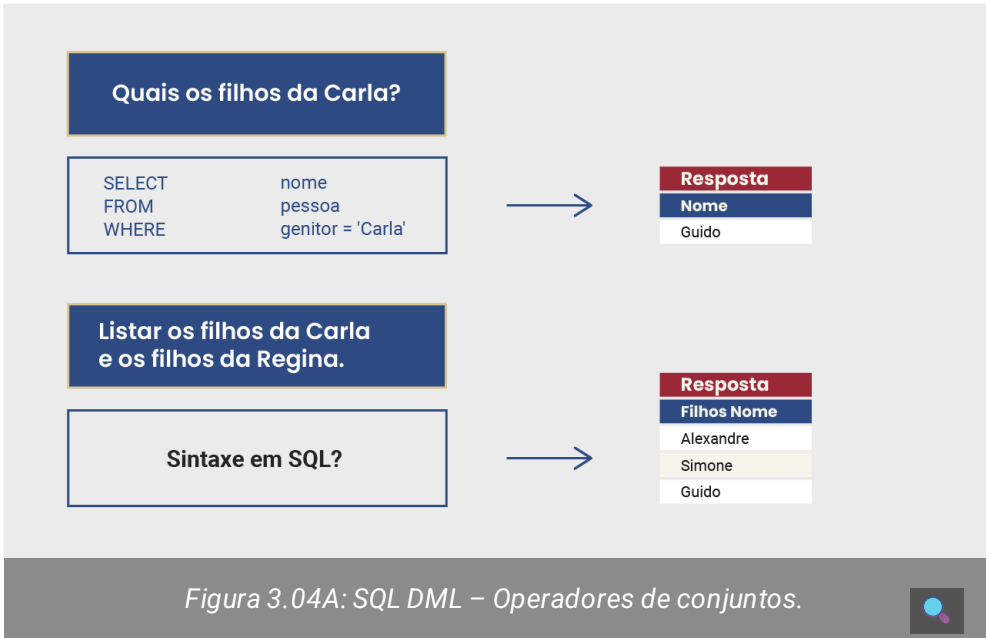
Se quisermos saber quais são os filhos de determinada pessoa, esta é uma consulta relativamente trivial em SQL, considerando a tabela Pessoa. No exemplo a seguir, estamos buscando os filhos da pessoa de nome Regina. Assim, segundo a instância da tabela Pessoa, mostrada na figura anterior, temos como resposta o Alexandre e a Simone.



Em SQL, podemos expressar essa consulta usando todas as cláusulas consideradas até o momento, como mostrado na figura a seguir. Veja que não precisamos da ORDER BY, mas, para fins didáticos, utilizamos a ordenação nesse exemplo. Para conhecer outros filhos de outras pessoas, basta substituir o valor na cláusula WHERE por outros nomes presentes na tabela Pessoa.



Por exemplo, na figura a seguir, temos uma consulta semelhante (“Quais os filhos da Carla”) e que respondemos como fizemos para a pessoa de nome Regina, simplificando apenas a expressão ao retirar a exigência de ordenação na tabela Resposta, desnecessária nesse caso. Em seguida, propomos uma consulta que pede para listarmos as soluções das duas últimas consultas simultaneamente, a saber, os filhos da Regina e os filhos da Carla, todos em uma mesma tabela.



Também não é uma consulta considerada difícil de expressar com o conhecimento básico da linguagem SQL: basta uma disjunção (OR) que resolvemos o que se pede! É exatamente o que mostramos na Figura 3.04B, na solução SQL#1 proposta.



Entretanto, aproveitamos esse exemplo simples para introduzir um novo operador na linguagem SQL oriundo da teoria de conjuntos. Como sabemos, relações e conjuntos têm muito em comum. Nesse caso, a solução SQL#2, que permite listar tanto os filhos da Carla como os filhos da Regina, utiliza a união de tabelas através do operador UNION, que tem a mesma interpretação do operador de união para conjuntos. De fato, na primeira parte da expressão SQL#2 da figura mostrada anteriormente, temos o “conjunto” dos filhos da Regina e, após a palavra UNION, temos o outro “conjunto” dos filhos da Carla.



Assim como para conjuntos, não podemos unir “laranjas com bananas”: é preciso que sejam duas tabelas com a mesma quantidade de colunas e, para cada coluna em sua posição relativa, que sejam domínios de valores compatíveis. Isso é o que se costuma chamar de “compatibilidade de tipo” (*type compatibility*).

No exemplo que estamos descrevendo, as expressões SQL geram duas tabelas com apenas uma coluna, ambas chamadas “Nome”, satisfazendo o mesmo domínio de atributos, que, na Figura 3.04B, diz respeito a nomes de pessoas. Se fossem “nomes de bares” com “marcas de cerveja”, a união também funcionaria, pois não deixam de ser conjuntos de caracteres alfabéticos como domínios. Entretanto, semanticamente falando, poderia não fazer sentido. Cabe ao programador evitar situações que permitem a realização da operação, mas que não têm interpretação adequada.



Aprenda mais

Veremos, a seguir, outros operadores tradicionais de conjuntos que se aplicam muito bem às relações. Apenas para podermos variar os exemplos, consideraremos agora as instâncias das tabelas Gosta e Vende.

Por um lado, temos as pessoas e as cervejas por elas apreciadas; por outro lado, as cervejas que são vendidas em quais bares e por qual valor.

GOSTA		VENDE		
Nome	Cerveja	Bar	Cerveja	Valor
Mariana	Antarctica	Depósito	Leffe	15.0
Alberto	Leffe	Pires	Jupiler	22.0
Alberto	Antarctica	Padoca	Antarctica	7.0
Diogo	Jupiler	Pires	Brahma	10.0
Alberto	Brahma	Padoca	Spaten	10.0
Carla	Spaten	Depósito	Brahma	11.0
Carla	Jupiler	Padoca	Patricia	18.0
Diogo	Leffe	Padoca	Brahma	10.0

Vamos estudar agora uma nova consulta, em que buscamos cervejas apreciadas por uma pessoa (**Carla**) que outra pessoa também aprecia (**Diogo**). Observando os dados ilustrados na instância da tabela Gosta, podemos obter a resposta, que, no caso, é a cerveja de marca **Jupiler**.

GOSTA		Resposta	
Nome	Cerveja	Cerveja	
Mariana	Antarctica	Jupiler	
Alberto	Leffe		
Alberto	Antarctica		
Diogo	Jupiler		
Alberto	Brahma		
Carla	Spaten	Jupiler	
Carla	Jupiler		
Diogo	Leffe		

Entre as cervejas de que a Carla gosta, quais são aquelas de que o Diogo também gosta?



De quais cervejas a Carla gosta?

```
SELECT DISTINCT Cerveja
FROM Gosta
WHERE Pessoa = 'Carla'
```

Como expressar em SQL?

Novamente, com os conhecimentos adquiridos na nossa primeira aula, podemos expressar as consultas em SQL que retornam as cervejas apreciadas pela Carla e pelo Diogo. Veja que as consultas são idênticas, só mudando o nome da pessoa.

De quais cervejas o Diogo gosta?

```
SELECT DISTINCT Cerveja
FROM Gosta
WHERE Pessoa = 'Diogo'
```

Resposta

Cerveja
Spaten
Jupiler

Resposta

Cerveja
Jupiler
Leffe

Para obter, por meio de uma única consulta, os valores obtidos em separado, temos a solução SQL #1 que utiliza o operador de pertinência **IN**. Na consulta mais externa, temos as cervejas do Diogo e, na subconsulta, as cervejas da Carla. Poderíamos, nesse caso, inverter e colocar o Diogo na subconsulta e a Carla na cláusula **SELECT** principal, pois retornaria resultado idêntico.

Podemos obter a mesma solução usando o operador **INTERSECT**. Como no caso do **UNION**, precisamos garantir a compatibilidade de tipo. Em termos de semântica, temos a mesma interpretação da teoria de conjuntos, que diz que o resultado da interseção contém os valores presentes em ambas as tabelas.

SQL#1:

```
SELECT DISTINCT Cerveja
FROM Gosta
WHERE Pessoa = 'Diogo'
AND Cerveja IN
      (SELECT Cerveja
       FROM Gosta
       WHERE Pessoa = 'Carla')
```

Básico: subconsulta com IN

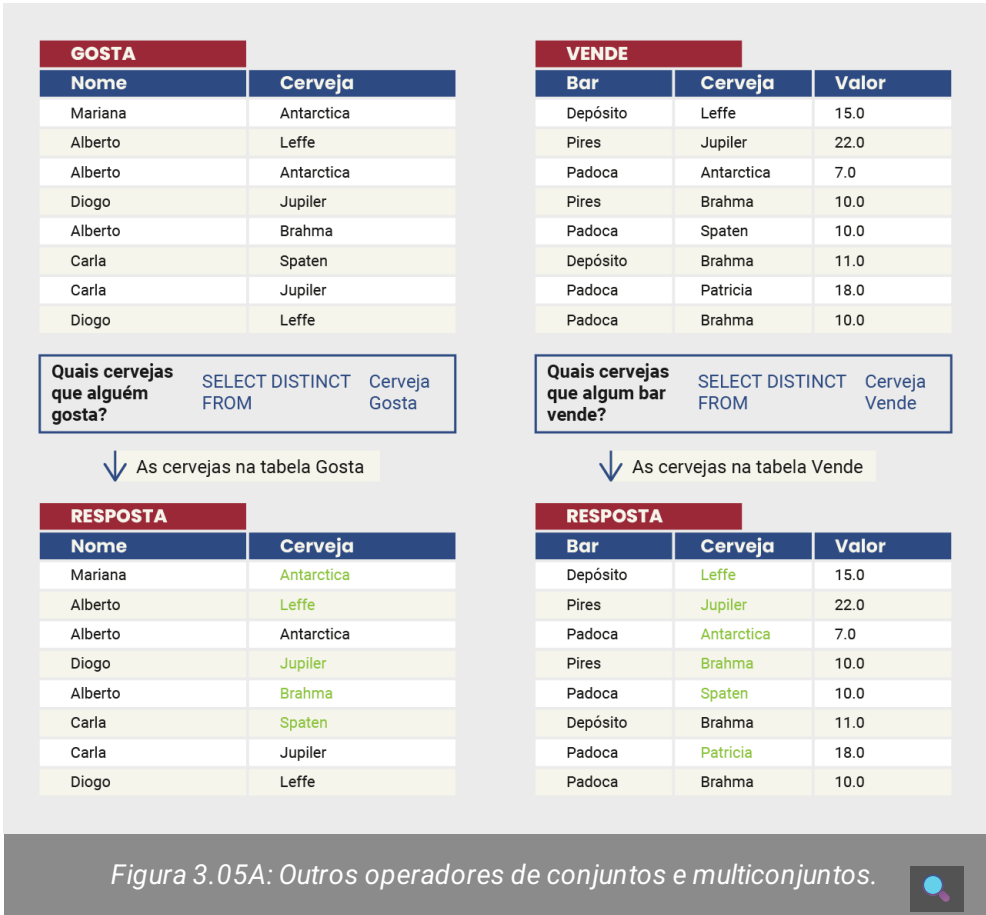
SQL#2:

```
(SELECT DISTINCT Cerveja
 FROM Gosta
 WHERE Pessoa = 'Diogo')
INTERSECT
(SELECT DISTINCT Cerveja
 FROM Gosta
 WHERE Pessoa = 'Carla')
```

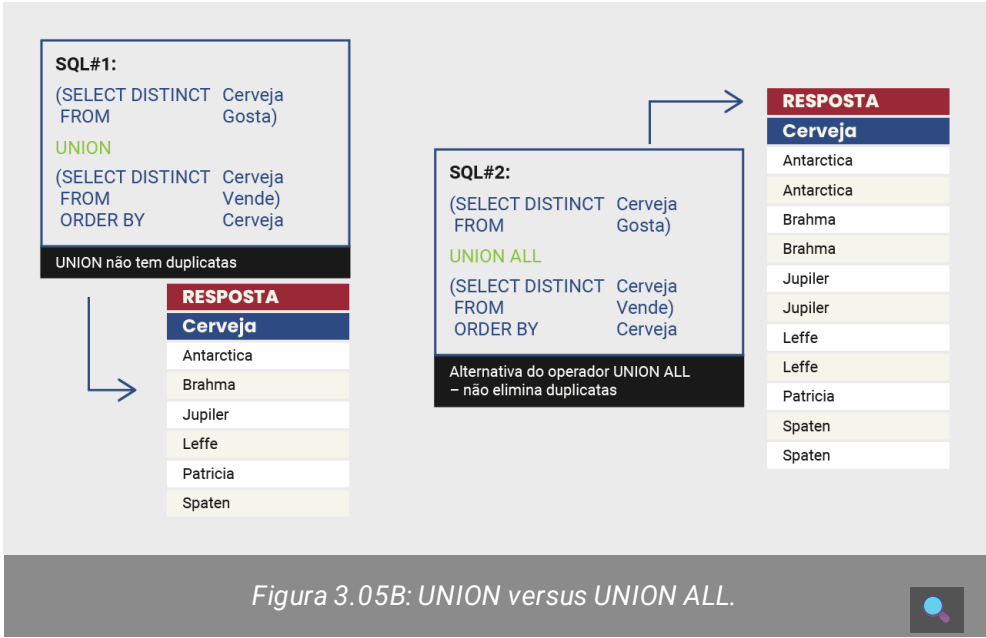
Alternativa: INTERSECT em SQL

Na prática, no entanto, nem sempre temos resultados que estão perfeitamente de acordo com a teoria. Algumas consultas podem retornar tabelas que representam multiconjuntos (*multisets*), que contêm elementos repetidos. Veja que, na figura a seguir, a consulta SQL que lista todas as cervejas na tabela Gosta poderia retornar alguns valores repetidos, como, por exemplo, Antarctica, Jupiler e Leffe. Isso só não vai ocorrer porque usamos um **DISTINCT** junto ao **SELECT** da expressão SQL. Algo similar ocorre se quisermos listar todas as cervejas vendidas em algum bar. Novamente, o uso do **DISTINCT** evita isso. Mas como fazer para responder em SQL uma

consulta que pede as cervejas que alguém gosta ou que algum bar vende?



Poderíamos usar novamente o operador UNION, que, na solução SQL#1 a seguir, retornaria todas as cervejas da tabela Gosta junto com as cervejas da tabela Venda. A operação de união elimina os valores em duplicata da solução, mas aqui a eliminação de valores repetidos nem precisa ser feita, pois as duas tabelas geradas pelas expressões em SQL usam o DISTINCT. Entretanto, em SQL, existe um operador adicional chamado UNION ALL, que, explicitamente, permite duplicatas. O motivo é direto e justificado: retirar valores repetidos em tabelas com grandes volumes é uma operação custosa, que pode levar muito tempo. Como a tabela com duplicatas tem um conteúdo de semântica similar ao da tabela sem duplicatas, sendo diferente apenas na quantidade de tuplas, muitas vezes usar o UNION ALL é a alternativa preferida em termos de desempenho. Cabe observar que existe também um operador INTERSECT ALL, que, de forma análoga, retorna uma relação fruto de uma interseção com valores repetidos.

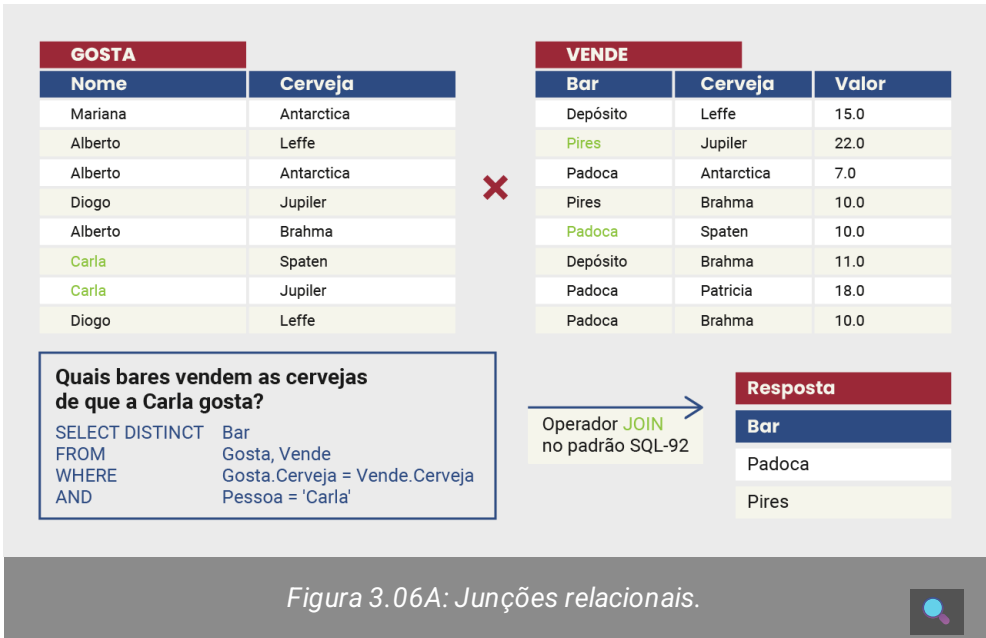




Todas as consultas em SQL até as aulas anteriores contemplaram apenas uma tabela de cada vez. Sabe-se que os bancos de dados relacionais na prática contêm duas ou mais tabelas que representam o mundo real. Assim, para podermos realizar consultas nesses bancos de dados naturalmente precisamos envolver duas ou mais tabelas do esquema relacional em questão na cláusula FROM. Quando isso ocorre, o SGBD relacional considera que todas as tuplas das tabelas presentes no FROM devem ser combinadas e comparadas antes de aplicarmos os predicados de restrição.

Se não há cláusula WHERE e, consequentemente, predicados que realizam o filtro das combinações de tuplas válidas, temos como resultado um produto cartesiano das relações constantes na cláusula FROM. Sabe-se que não há semântica limitada para produtos cartesianos em si, e, assim, estes devem ser evitados. Logo, temos no modelo relacional um operador chamado de “junção” (*Join*, em inglês), que, na prática, realiza um filtro sobre a combinação de tuplas nas tabelas do FROM atendendo às condições de comparação lógica dos predicados na cláusula WHERE. Nesses casos, necessariamente teremos pelo menos uma condição de comparação entre atributos para cada par de tabelas listadas na cláusula FROM.

Na figura 3.06A, temos um exemplo típico de uso do operador de junção para o esquema relacional das cervejas. Gostaríamos de expressar em SQL a consulta que retorna a lista de bares que vendem as cervejas de que determinada pessoa gosta.





Para essa consulta, precisamos utilizar duas tabelas do esquema do banco de cervejas. Assim, na cláusula FROM, temos tanto a tabela Gosta como a tabela Vende, e, na cláusula WHERE, uma comparação de igualdade entre os valores do atributo cerveja nas duas tabelas e uma restrição adicional no nome da pessoa (Carla) usando conjunção (AND). Observe que, como os atributos têm o mesmo nome em ambas as tabelas, é preciso referenciá-los com o nome da tabela antes, caso contrário o SGBD daria mensagem de erro por conta da ambiguidade.

Fique ligado

A junção pode ser interpretada da seguinte maneira: após combinar todas as tuplas de uma tabela com as tuplas da outra, a saída do operador corresponde apenas às tuplas que retornam como verdadeira a comparação de igualdade entre atributos que é explicitada. Ou seja, nesse caso, primeiro combinamos as tuplas de Gosta e Vende e filtramos para retornar apenas aquelas em que as cervejas têm o mesmo valor. Nesse caso, restringimos também em seguida para as tuplas com valor do atributo Pessoa igual a Carla. Finalmente, para as tuplas que atenderam essas restrições, exibimos apenas os valores da coluna Bar na saída.

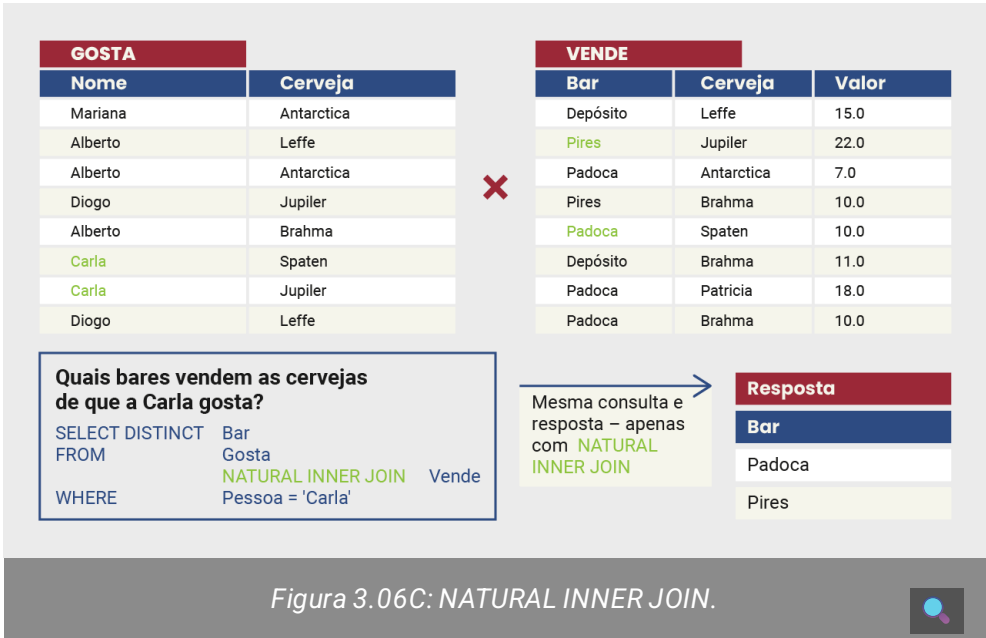
Essa junção é também conhecida como equijunção (*equijoin*, em inglês), pois a comparação de atributos nas tabelas da cláusula FROM utiliza o operador de igualdade. É o caso mais comum de junção, que permite juntar dados em tabelas distintas, mas que, juntas, compõem a informação completa que o banco de dados traz. Há outras comparações usando outros operadores, mas não são muito comuns na prática.

A linguagem SQL inclui um termo para explicitar o uso de junções, e isso é ilustrado na figura a seguir. Trata-se do INNER JOIN (junção interna), que ainda exige que fiquem explicitadas na consulta as colunas que serão comparadas para realizar a junção.



Na Figura 3.06C, temos mais uma versão dessa mesma consulta utilizando o operador de NATURAL INNER JOIN (junção natural). É uma equijunção em que a comparação de igualdade é feita sobre atributos de mesmo nome nas tabelas envolvidas. Isso acontece com tanta frequência que a simplificação sintática é bastante útil. Na

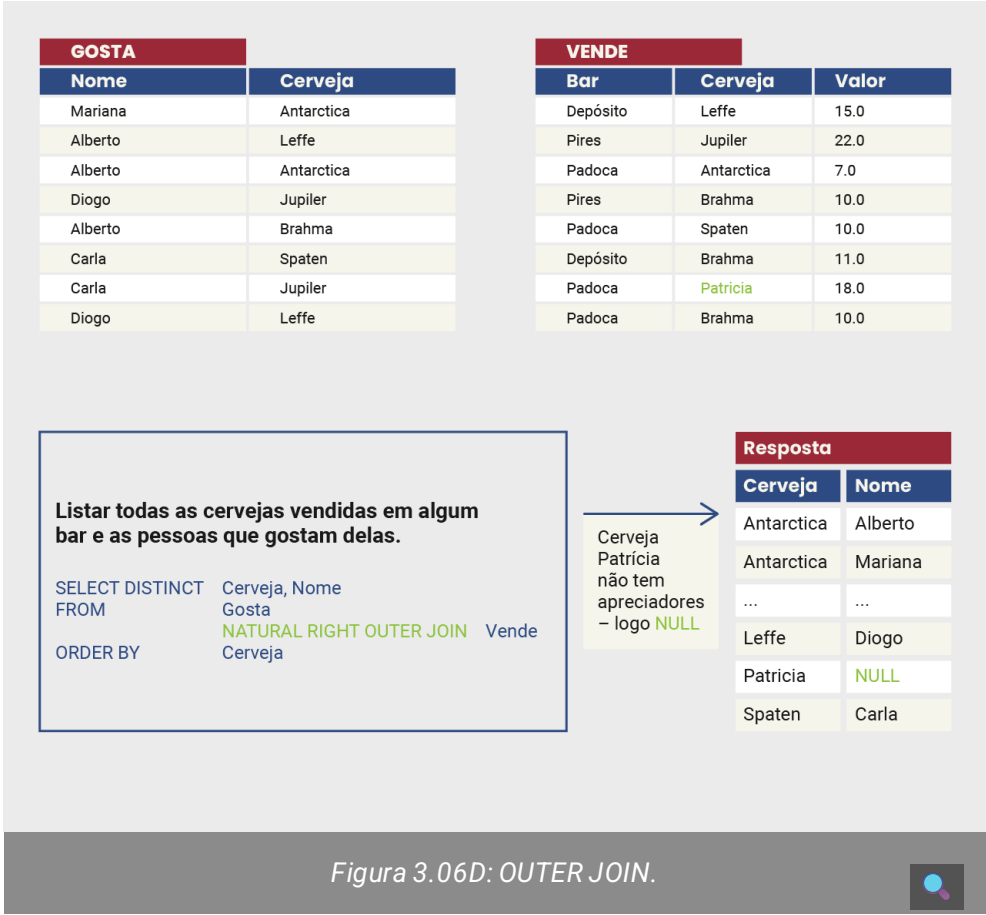
figura, vemos que a expressão fica bem mais simples do que as anteriores.



É importante, neste momento, alertar para o fato de que em muitos esquemas relacionais temos atributos de mesmo nome que não têm a mesma semântica ou interpretação na aplicação em questão. Por exemplo, no esquema relacional das cervejas da Figura 3.01, temos o atributo Nome para a tabela Pessoas e também para a tabela Bar. Normalmente não faria sentido essa comparação no caso da junção natural. É verdade que esses atributos podem até ter o mesmo domínio de valores. Nomes de bares e nomes de pessoas são, ambos, compostos de caracteres alfabéticos e podem, computacionalmente falando, ser comparados usando operadores da linguagem SQL. Seria o caso, por exemplo, de um manuseio pouco comum em uma consulta que queira saber se existe alguma pessoa nesse banco de dados que tenha o nome de algum bar.

A operação de junção interna também é normalmente chamada apenas de “junção”. Entretanto, existe outro tipo de junção, chamado de junção externa, e, por isso, a junção “normal” também é conhecida como interna. Essa junção externa (*OUTER JOIN*, do inglês) é similar à junção interna, mas considera, para fins de resultado na saída, as tuplas que se combinam na junção com outras que não se combinam.

No exemplo a seguir, temos uma consulta que pretende exibir uma lista de cervejas vendidas em algum bar e as pessoas que as apreciam. Como pode acontecer, em alguma instância, de termos cervejas para as quais não tenhamos pessoas que gostam, estas não apareceriam na junção interna. Entretanto, na junção externa são exibidas tanto as tuplas que satisfazem a condição da junção interna como as tuplas que não satisfazem. Nesse caso, complementamos o resultado com o valor NULO. Para o exemplo da base de dados de cervejas e a instância particular ilustrada, temos que não há pessoas associadas à cerveja Patricia. Logo, como não há como deixar a coluna Pessoa sem valor, é atribuído o valor NULO.



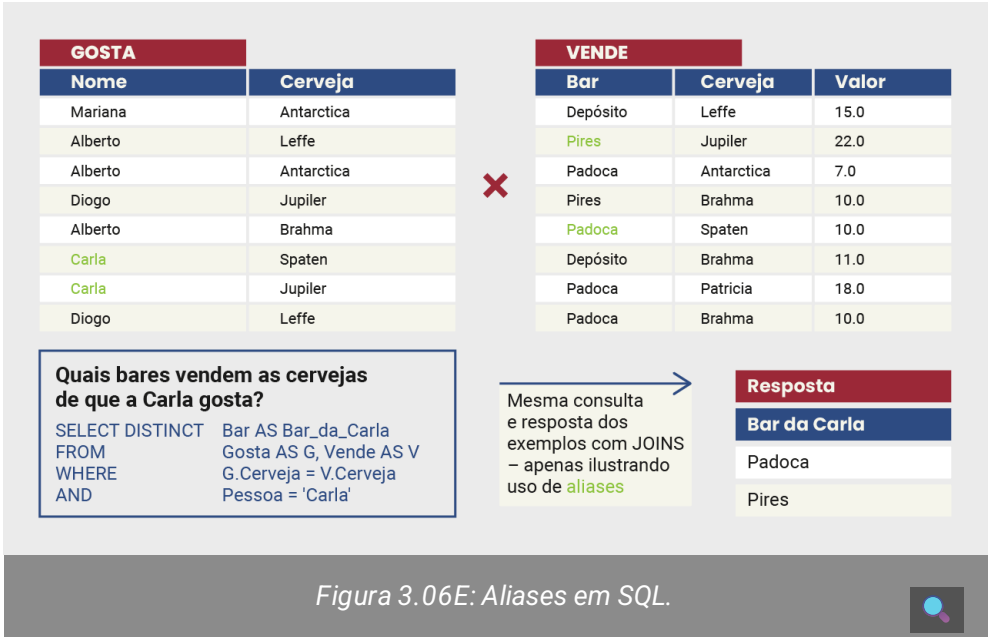
Existem algumas variações de junções externas. Nesse caso, como visto na figura 3.06D, utilizamos a junção externa à direita (*RIGHT OUTER JOIN*). O fato de ser à direita diz respeito apenas à posição relativa das tabelas na expressão da junção e para qual coluna é colocado o valor NULO na resposta. Por exemplo, se optássemos por começar pela tabela Vende, a resposta em SQL ficaria com a junção externa à esquerda (*LEFT JOIN*). Quando a junção externa permitir NULOS nas duas tabelas, ela é chamada de “junção externa completa”, ou *FULL OUTER JOIN*. Nesse exemplo particular, isso faria sentido se tivéssemos, além de cervejas para as quais não temos pessoas que as apreciam, cervejas de que algumas pessoas gostam, mas que não são vendidas em nenhum bar.

Alias ou apelidos para tabelas e atributos



As expressões de consultas em SQL envolvem, de maneira geral, muitas tabelas e diversos atributos. Não somente alguns atributos têm o mesmo nome, como já vimos, mas também podemos querer visualizar na resposta de consulta as colunas de uma tabela com nomes mais apropriados para a compreensão.

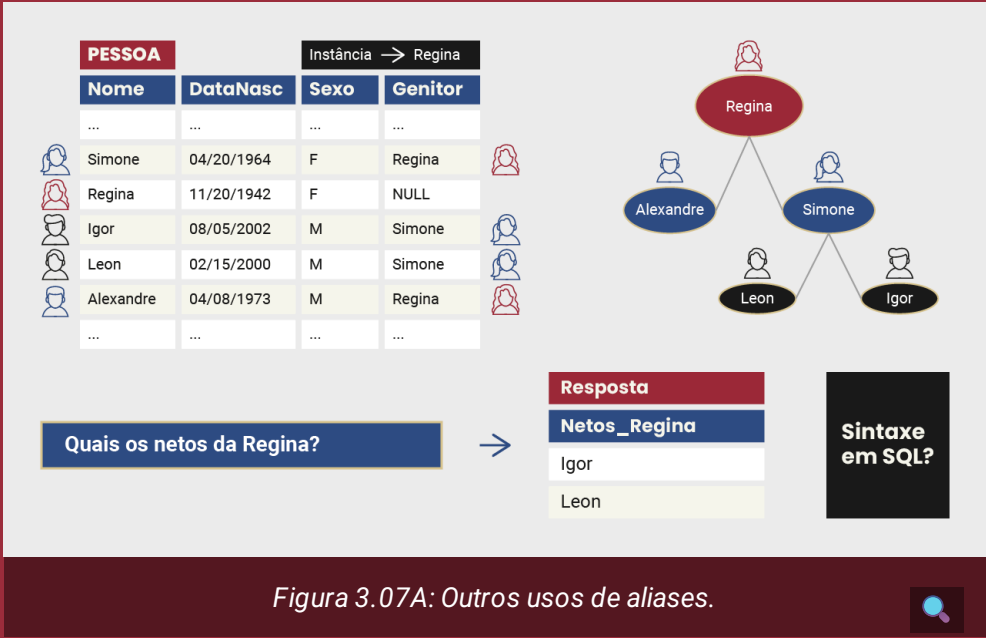
Assim, a linguagem SQL permite o uso de *aliases* (ou apelidos) tanto para desambiguar atributos como para simplificar a maneira de expressá-los nas consultas. A figura a seguir traz um exemplo duplo de utilidade dos *aliases*. Na cláusula SELECT, a coluna na tabela de resposta terá um nome bem mais claro e apropriado (Bar_da_Carla). Além disso, as tabelas Gosta e Vende são renomeadas para G e V, respectivamente. Assim, a expressão de comparação na cláusula WHERE também é facilitada.



Explora

Vamos explorar um pouco mais o uso de *aliases* nas expressões SQL. Um pouco antes, nesta aula, resolvemos a consulta de obter os filhos de alguma pessoa, por exemplo, os filhos da Regina. Como faríamos agora para obter os netos da Regina?

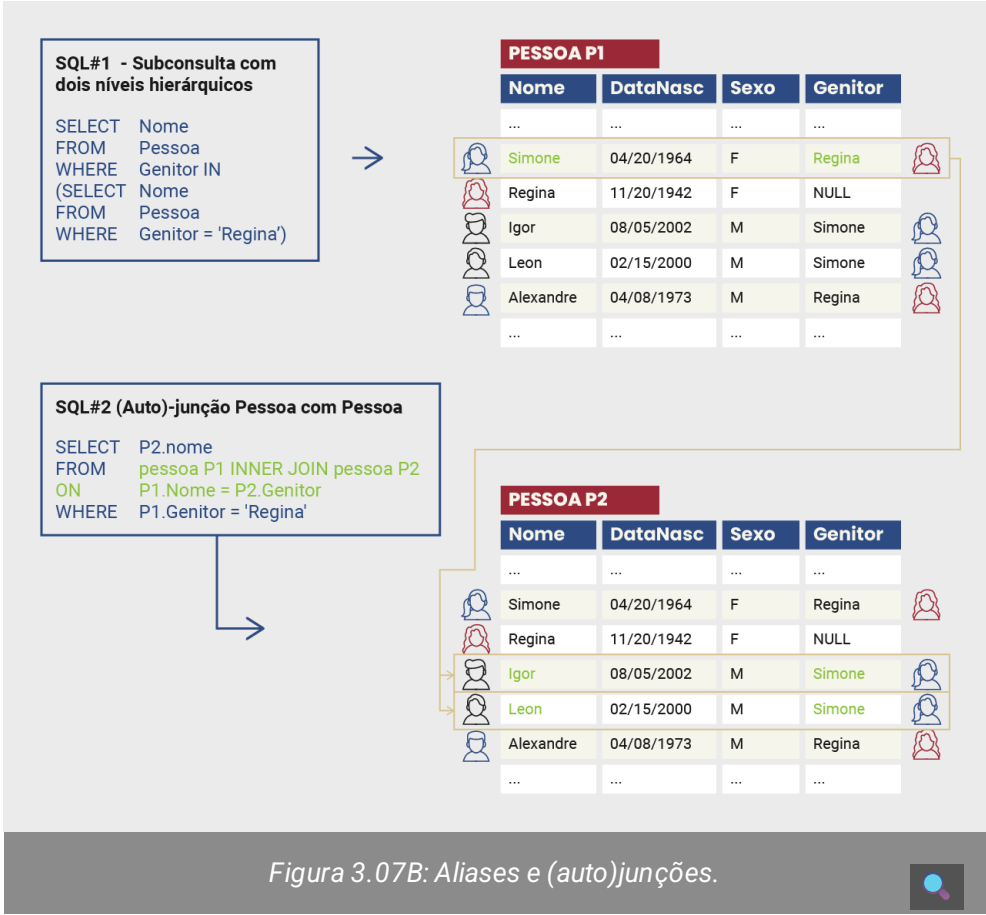
A tabela Pessoa (Figura 3.02) só fala de genitores (mães ou pais) e filhos diretamente, e, por isso, a solução na Figura 3.03B era imediata e simples. Agora teremos de manusear com linguagens relacionais para obter o resultado desejado com a devida interpretação sobre hierarquias familiares, como Avós, Pais, Filhos, Netos, Bisnetos, etc.



Aqui temos uma parte da instância da tabela Pessoa, com uma representação gráfica auxiliar da hierarquia familiar ali representada, envolvendo a pessoa de nome Regina. Olhando diretamente para os dados, temos como resposta que os netos da Regina são o Igor e o Leon. Mas como expressar uma consulta em SQL que permita retornar tais valores?

Uma alternativa seria “navegar” pelos dados de acordo com a semântica de uma hierarquia familiar. Para essa consulta, são apenas (e exatamente!) dois níveis na árvore, como demonstrado na figura anterior: Regina é mãe do Alexandre e da Simone; Alexandre não tem filhos, mas Simone sim: ela é mãe do Leon e do Igor. Consequentemente, Regina é avó do Leon e do Igor.

Na Figura 3.07B, temos a ilustração para essa estratégia de solução, com SQL convencional e básico, usando uma subconsulta para auxiliar na expressão. Na expressão SQL#1, temos que resolver, primeiro, a consulta SQL que retorna os filhos da Regina. De posse dessa tabela auxiliar representada pela subconsulta, a consulta mais externa pode comparar genitores com os valores na tabela dos filhos da Regina (Alexandre e Simone). Os eventuais filhos destes estarão na resposta da expressão SQL, e, como a Simone tem dois filhos, estes vão compor a tabela resposta.

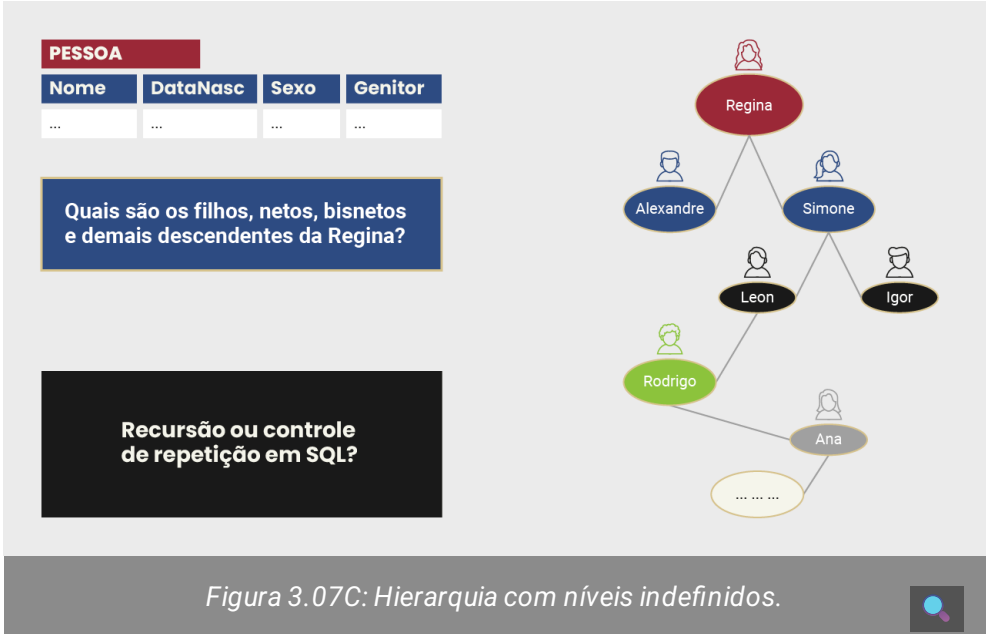


Entretanto, temos outra estratégia usando junções para essa mesma consulta. Na realidade, é uma (auto)junção, pois se trata de uma junção de uma tabela (Pessoa) com ela mesma. Seria como se tivéssemos duas “cópias” da tabela Pessoa, mas, em cada “cópia”, os atributos passam a ter “papéis” diferentes. Na cópia P1, temos a pessoa Regina como avó e a pessoa Simone como filha; já na cópia P2 a Simone aparece como genitora (mãe) do Igor e do Leon, que, naturalmente, serão netos da Regina. Na figura mostrada anteriormente, ilustramos graficamente a situação, que permite compreender os efeitos da expressão SQL #2.

Fique ligado

Algumas observações relevantes: em primeiro lugar, para (auto)junções, os *aliases* são obrigatórios. Sem usar *aliases* não teríamos como diferenciar as “cópias” da tabela Pessoa. Em segundo lugar, veja que podemos, para alguns SGBDs, não usar “AS” para os *aliases*, bastando um espaço em branco como separador. No exemplo, temos “Pessoa P1” em vez de “Pessoa AS P1”, o que também segue válido na linguagem SQL, claro. A maioria dos SGBDs permite isso; caso contrário, o “AS” deve ser mantido. Por fim, a estratégia que usa (auto)junção pode ser considerada mais declarativa, pois, diferentemente da primeira solução, não há necessariamente uma ordem de execução, em que a subconsulta é executada primeiro. Logo, o otimizador tem mais possibilidades de execução da consulta e gera um *Query Execution Plan* (QEP) mais adequado e, provavelmente, mais eficiente.

Poderíamos seguir explorando a hierarquia familiar e utilizar SQL para retornar qualquer grau familiar conhecido. Se quisermos, além dos netos, os bisnetos, basta realizar mais uma operação de autojunção na tabela Pessoa, pois são três níveis hierárquicos. Na figura, teríamos na resposta da consulta a pessoa de nome Rodrigo como bisneto da Regina, pois é filho do Leon, que é seu neto já que é filho da Simone, por sua vez, filha da Regina.

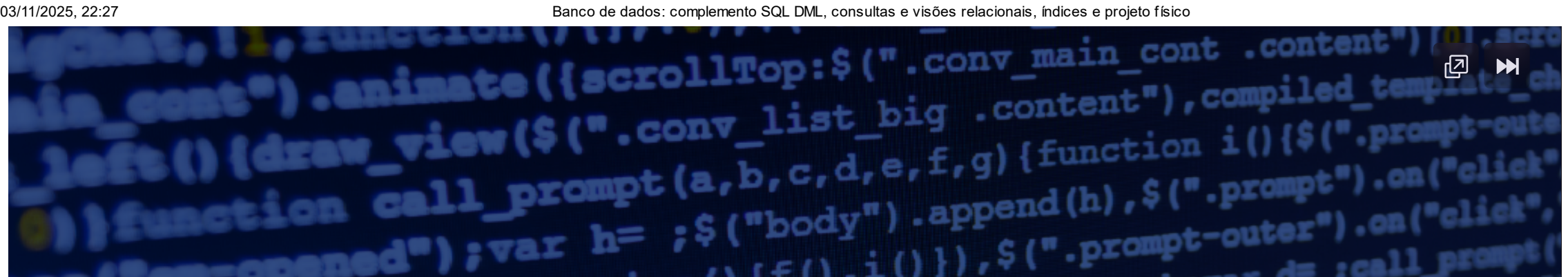


E se quisermos todos os descendentes da Regina? Na figura 3.07C, ilustramos essa situação com a árvore da hierarquia familiar, mas, se não sabemos quantos níveis temos na hierarquia, como especificar a junção? A mesma dúvida também pode ser colocada para a solução SQL#1 que utiliza subconsultas: se não sabemos quantos níveis hierárquicos existem envolvendo descendentes da Regina, em que momento paramos de criar subconsultas que permitem buscar por netos ou tataranetos?

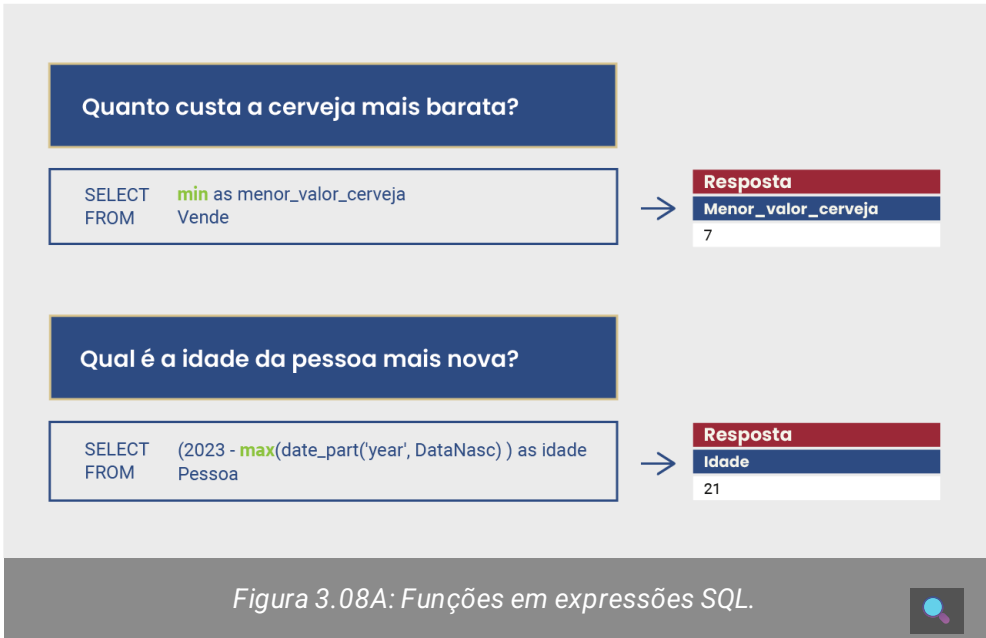
Este é um dos limites teóricos da linguagem SQL, pois trata-se de uma linguagem baseada na lógica da 1ª ordem, o que significa que não temos nem controles de repetição, como expressões WHILE ou FOR das linguagens de programação mais conhecidas, nem sintaxe para expressar recursão.

Para resolver consultas desse tipo, sem saber quantos níveis hierárquicos precisamos percorrer, em princípio é necessário combinar SQL com linguagens de programação, como **C** ou **Python**. Existem iniciativas para incluir expressões recursivas em SQL, que poderiam resolver o problema, como é o caso da cláusula WITH com tabelas temporárias recursivas, porém é mais um artifício sintático do que aumento do poder de expressão da linguagem.

Uso de funções em tabelas

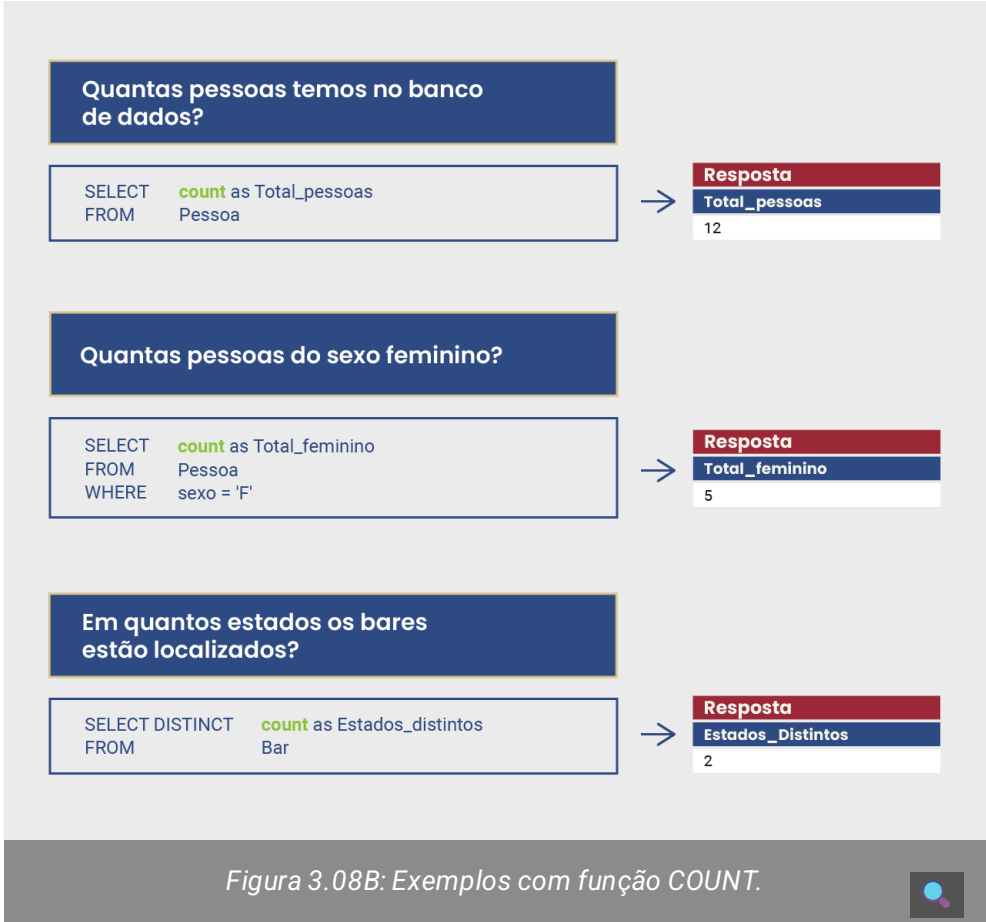


Quando aprendemos a trabalhar com o modelo de dados relacional, em que temos todos os dados representados em tabelas, muitos se lembram das planilhas eletrônicas de programas como o Excel (da Microsoft) ou Sheets (do Google). No caso de SQL, também temos funções que permitem estatísticas simples sobre conjuntos de valores nas colunas de cada tabela. Podemos citar os operadores MAX (maior valor), MIN (menor valor), AVG (valor médio) e SUM (soma de valores), que atuam sobre os multiconjuntos de valores nos atributos de uma relação e retornam sempre um único valor.



Nessa figura, temos um exemplo do uso do MIN para saber qual o valor da cerveja mais barata à venda em algum bar. Estamos considerando aqui as instâncias da tabela Vende da Figura 3.01, cujo menor valor de cerveja à venda é de R\$ 7,00. Já para a instância da Figura 3.02, temos que o ano mais recente é 2002. Tendo como referência o ano de 2023, a pessoa mais nova tem, ou completa, 21 anos de idade. Usamos, nesse caso, funções particulares aplicáveis ao tipo de dados DATE, como ocorre para o atributo DataNasc. É possível extrair o ano de uma data de nascimento, permitindo fazer contas com anos.

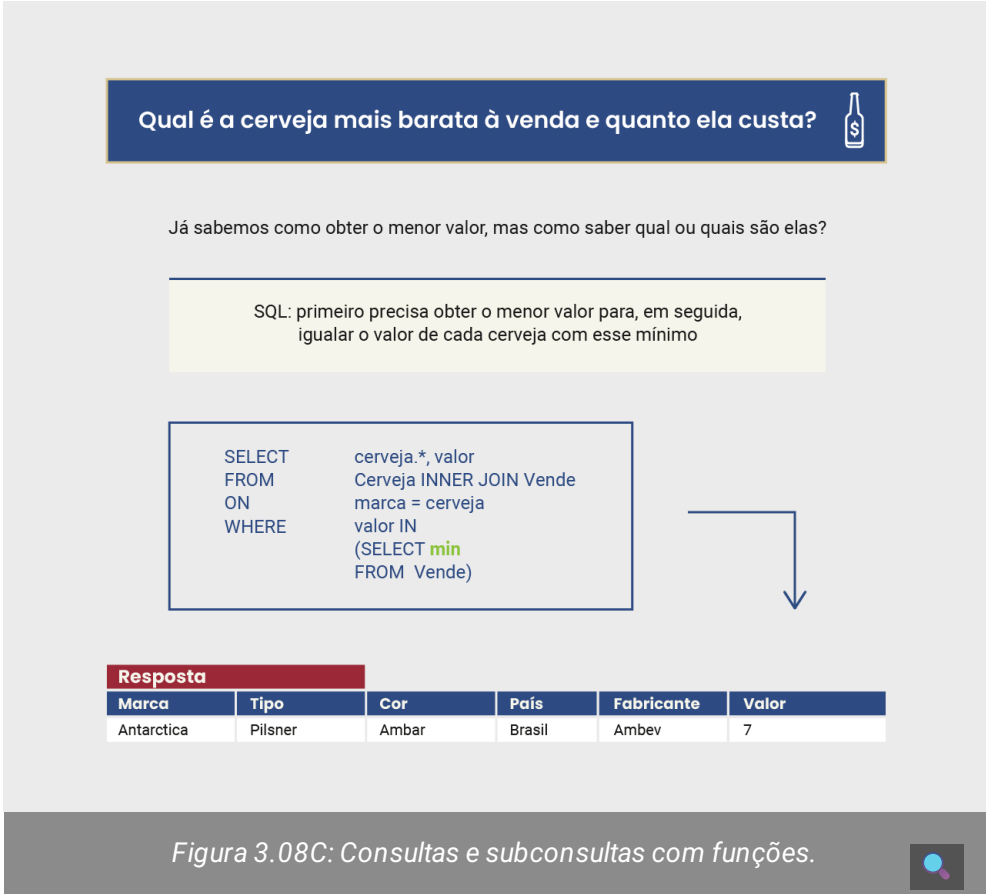
De maneira geral, temos funções que se aplicam a dados numéricos. Na figura que segue, exemplificamos algumas utilizações do operador COUNT, que literalmente conta quantas tuplas existem na tabela referenciada, de acordo com as restrições impostas na cláusula WHERE. Cabe observar que NULOS são ora ignorados, ora considerados como zero nas contas realizadas. Mais: se não queremos contar valores repetidos, basta usar o DISTINCT junto com o COUNT, como ocorre na última consulta exemplificada na figura.



As consultas da figura anterior são válidas e podem ser executadas em qualquer SGBD relacional. No entanto, sabemos que a metabase ou catálogo do banco de dados muitas vezes já mantém os valores atualizados disponíveis para consulta, como é o caso do total de pessoas na tabela Pessoa ou mesmo o total de pessoas de sexo masculino e feminino. Como normalmente temos bancos de dados volumosos, contar quantidades de tuplas de tabelas persistidas pode ser ineficiente.

Mesmo com uso das funções, sabe-se que a estratégia de manuseio de relações, ou tabelas, visando a resolver consultas aparentemente simples, nem sempre é trivial ou imediata de se expressar na linguagem SQL.

Na Figura 3.08A, foi apresentada a expressão para o valor da cerveja mais barata, mas como saber qual cerveja é vendida por esse valor? Na figura a seguir, por exemplo, estamos buscando obter a marca e demais informações da cerveja mais barata à venda.



obter dados de cervejas além da marca) e a tabela Vende, comparando o valor da coluna Marca com a coluna Cerveja. Nesse exemplo, estamos aproveitando para ilustrar um aspecto sintático utilizado na cláusula SELECT. De fato, em vez de listar todas as colunas da tabela Cerveja, usamos “cerveja.*”, que facilita a expressão (para todas as colunas de uma das tabelas no FROM) para visualização da resposta da consulta. Esse artifício de uso do asterisco para representar todas as colunas de apenas uma das tabelas na cláusula SELECT pode ser usado em qualquer consulta SQL.



Funções e agregados: cláusula *Group By*

Se continuarmos buscando utilizar funções em consultas SQL, podemos ter dificuldades quando misturamos, na cláusula SELECT, funções com atributos simples.

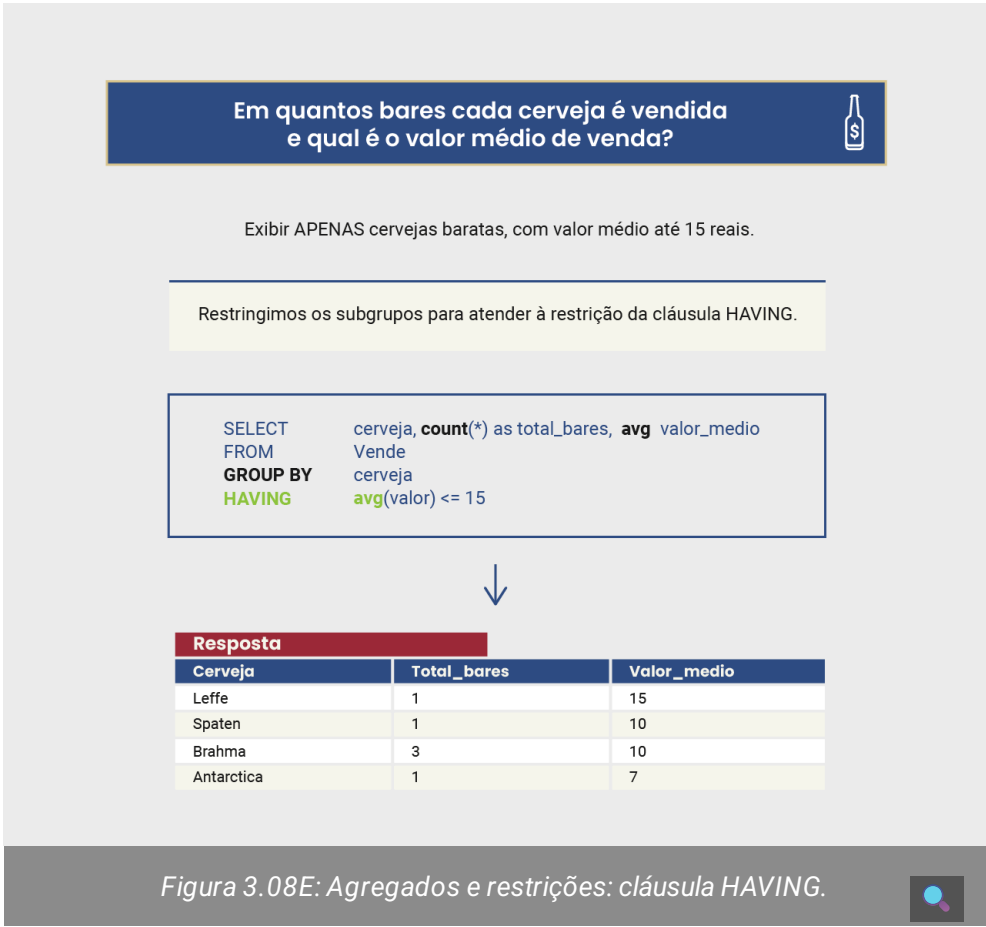
Por exemplo, para obter uma lista de cervejas, mostrando para cada uma em quantos bares ela é vendida, e por qual valor médio de venda, não basta simplesmente “fazer um SELECT” em que associamos as cervejas aos seus valores médios de venda, e à quantidade de bares em que são vendidas, de acordo com a relação Vende. Se estamos calculando o valor médio de venda de todas as cervejas na tabela Vende, qual das cervejas mostraríamos ao lado desse valor por conta da cláusula SELECT?



Para isso, a linguagem SQL sugere utilizar uma nova cláusula, **chamada GROUP BY**. Como ilustrado na figura anterior, será possível agrupar tuplas com valores iguais em alguns atributos e considerar estatísticas para cada grupo de tuplas em separado. Nesse caso, todas as tuplas com valor igual para nomes de cerveja vão compor um grupo. Apenas a cerveja de marca Brahma é vendida em mais de um bar nessa instância. Assim, para as demais cervejas, o valor na tabela Vende é igual ao valor médio, sempre de acordo com a instância da Figura 3.01.

Temos ainda a possibilidade de restringir ainda mais as tuplas que serão exibidas por atenderem à definição de uma consulta SQL envolvendo agrupamentos. Basta usar **uma nova cláusula (HAVING)** que permite expressar consultas que restringem valores sobre os grupos formados no Group By. A cláusula HAVING é um filtro nos grupos, assim como a cláusula WHERE é um filtro nas tuplas das tabelas.

A figura a seguir ilustra essa situação, em que estendemos a consulta discutida anteriormente para mostrar apenas cervejas consideradas baratas, com valor médio de venda de até 15 reais. A consulta em SQL é a mesma da Figura 3.08D, com uma cláusula HAVING adicional para não considerar grupos com valor médio de venda nos bares do banco de dados maior que 15 reais. Algumas tuplas exibidas na Figura 3.08D não mais aparecem na resposta por conta desse filtro HAVING. É importante observar que só podemos ter cláusula HAVING na presença de cláusula GROUP BY.



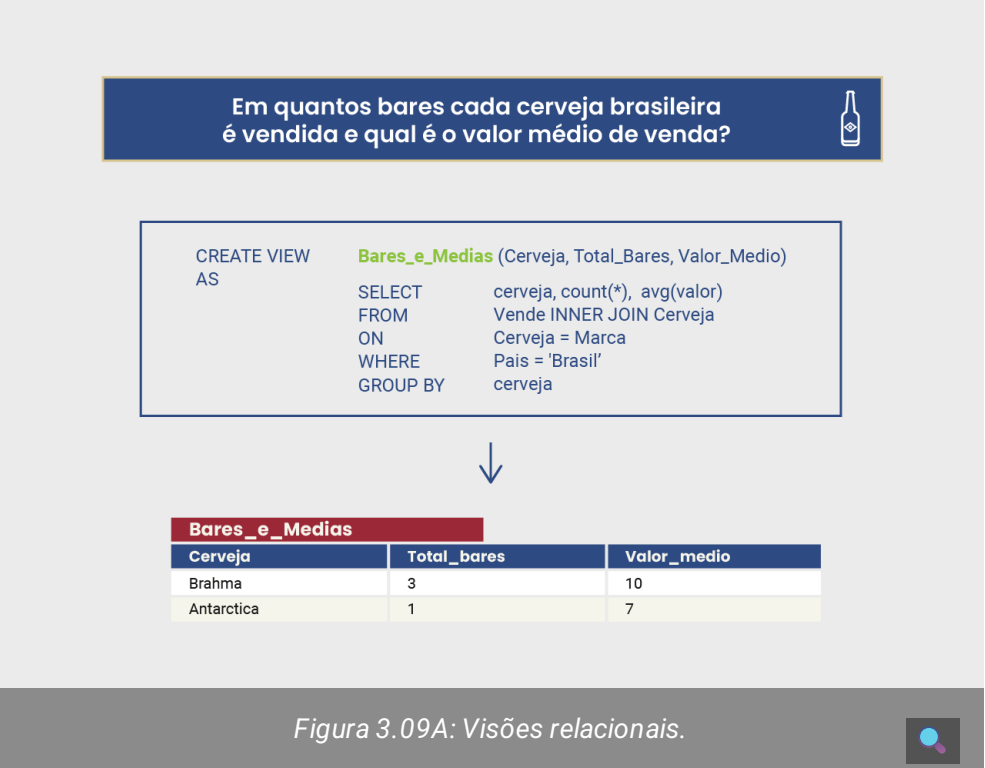
Neste momento, podemos dizer que já teremos visto todas as cláusulas importantes da linguagem SQL (SELECT / FROM / WHERE / GROUP BY / HAVING / ORDER BY), com algumas variações que permitem explorar todo o poder de expressão de linguagens relacionais, e conhecer seus limites. Vamos agora explorar outros objetos que podem ser criados, e práticas aplicadas ao modelo de dados relacional, para facilitar o manuseio de esquemas relacionais de maneira geral e buscar bons desempenhos na execução de consultas.

Visões relacionais



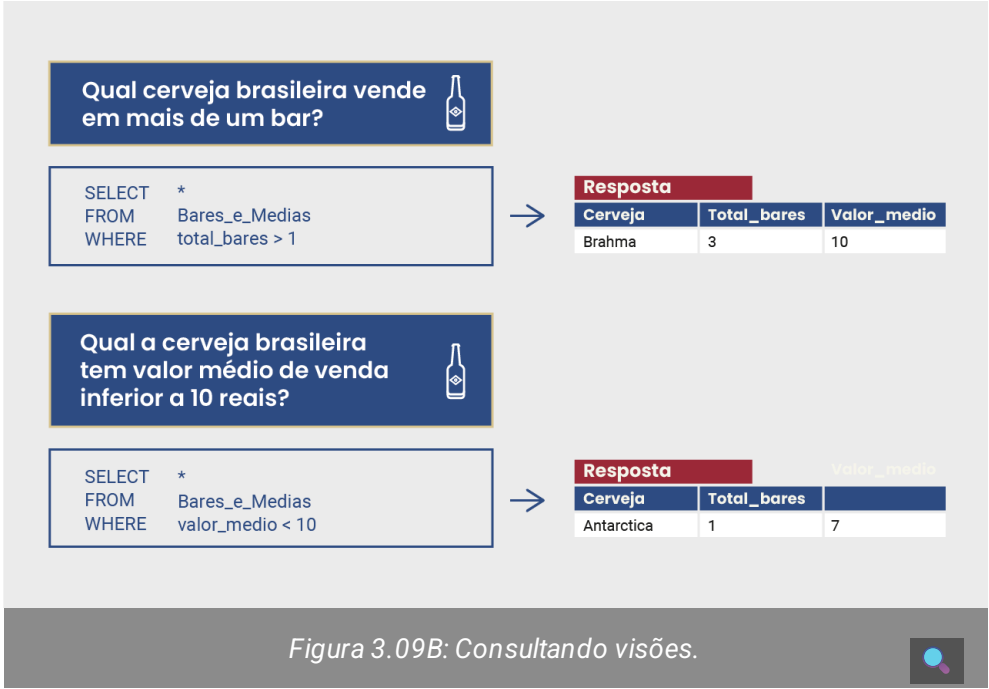
Na Figura 3.08D, explicamos como resolver em SQL uma consulta que exibe, para cada cerveja, em quantos bares ela é vendida e por qual valor médio. Se essa consulta é executada diversas vezes ao dia, por mais de um usuário, poderíamos de alguma maneira “salvar” essa consulta para poder reaproveitá-la outras vezes.

Existe um **objeto conhecido como visão** (ou *view*, em inglês), que permite guardar no catálogo do banco as definições de consultas para uso posterior. Na figura a seguir, ilustramos a criação de uma visão chamada Bares_e_Medias (com comando CREATE VIEW), que armazena a expressão da consulta que havíamos discutido anteriormente. Se executássemos a consulta que define a visão, obteríamos as duas tuplas exibidas na mesma figura, fruto de um acesso à visão como se a mesma fosse uma tabela, como Vende ou Cerveja.



Veja que uma visão guarda apenas a expressão SQL, não os dados que são resultado da execução da consulta. Por isso, muitas vezes as visões são chamadas de tabelas virtuais, pois os dados não são armazenados, apenas a consulta que define a visão. Uma vez definida uma visão, esta pode ser usada para definir outras visões, e assim por diante.

Entre outras vantagens, a visão Bares_e_Medias pode ser manuseada exatamente como uma tabela, ocultando a expressão em SQL relativamente complicada para leigos na linguagem. É o que ilustramos na figura a seguir, em que realizamos consultas sobre a visão como se fosse mais uma tabela no esquema relacional das Cervejas. Se for mesmo uma consulta muito frequente, também temos por vantagem o fato de a expressão da consulta referente à visão já estar pré-compilada e com seu plano de execução ótimo pré-definido.



As visões também podem ser usadas para ocultar atributos (ou colunas) presentes em tabelas persistidas no banco de dados, que não podem ser disponibilizadas, ou mesmo conhecidas, por uma parte dos usuários do banco. No caso da visão Bares_e_Medias, os detalhes de cada bebida, presentes na tabela Cerveja, não são exibidos, apenas a marca.

Também podemos considerar as visões para controle de integridade dos dados em um banco relacional. Nesse caso, além de criar a visão com o comando CREATE VIEW e o resto da expressão em SQL que define a consulta, é preciso adicionar a cláusula WITH CHECK OPTION ao final. Se fizéssemos isso com a visão Bares_e_Medias, e apenas essa visão fosse acessível aos usuários em geral, apenas cervejas brasileiras seriam aceitas no banco de dados por conta da restrição da cláusula WHERE igualando País ao valor “Brasil”. Esse controle seria feito diretamente no banco de dados, como parte da definição do esquema, e não precisaria mais ser incluído como trecho de código em programas e aplicações diversos.

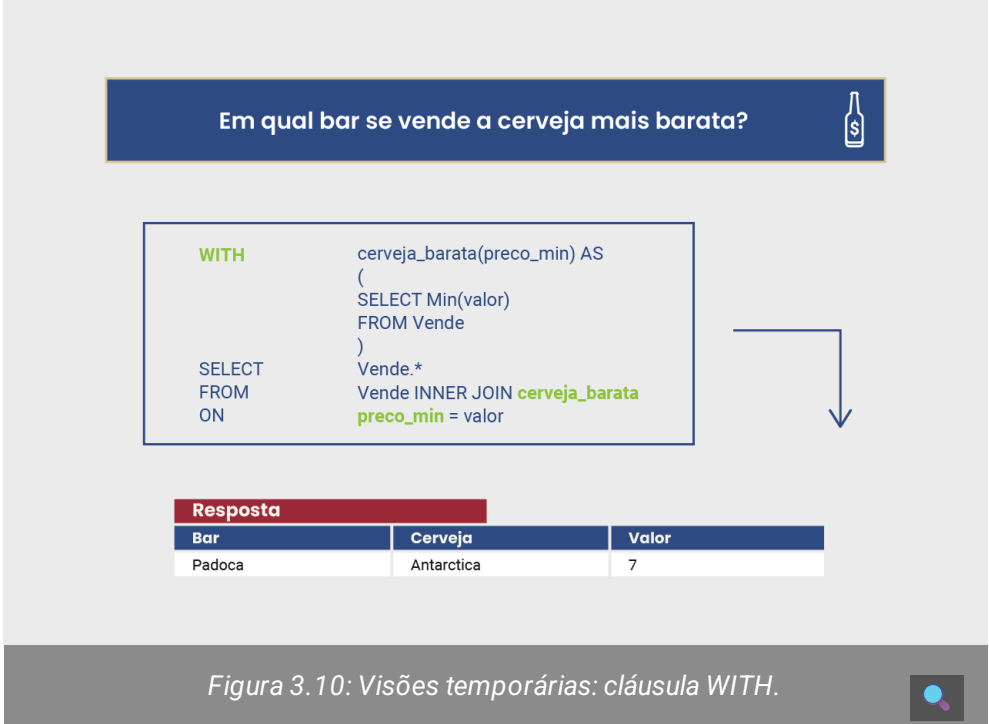
Visões relacionais

Visões também podem ser usadas como interface para atualização de dados no banco. Há limites, mas, em alguns casos práticos, é possível trabalhar apenas com visões, evitando dar acesso direto às tabelas que estão persistidas no banco de dados, reduzindo possíveis maneirismos não autorizados. Assista ao vídeo a seguir sobre visões, em que utilizamos um exemplo de esquema relacional simplificado das cervejas, chamado Beer, com apenas três tabelas com nomes similares, mas não exatamente com os mesmos esquemas relacionais.





Existem também visões temporárias, que são válidas apenas para consultas específicas, mas que mesmo assim são muito usadas na prática. Para isso, precisamos usar a cláusula WITH, como ilustramos na figura a seguir.



Já tínhamos visto que é simples exibir o valor da cerveja mais barata, porém um pouco mais complicado expressar em SQL consultas que necessitam desse menor valor de venda nas suas soluções. Nesse exemplo, com a cláusula WITH, podemos definir uma visão temporária chamada `cerveja_barata`, que tem apenas uma coluna – o preço mínimo –, e utilizar essa “tabela virtual” na expressão da consulta em seguida que faz um INNER JOIN entre a tabela `Vende` e `Cerveja_Barata`.

Quando comentamos sobre limites da linguagem SQL com respeito a não contemplar controle de repetição ou recursão, como é o caso da consulta que deseja exibir todos os nomes em uma hierarquia familiar, algumas implementações em SGBDs relacionais conseguem contornar esse problema exatamente com a cláusula WITH. Como se trata de um assunto complexo e extenso, recomendamos ao aluno consultar as referências bibliográficas sugeridas para entender como consultas recursivas podem ser executadas com apoio da cláusula WITH.

Índices em bancos de dados relacionais



Terminaremos esta aula introduzindo rapidamente o conceito de índices em bancos de dados relacionais. Assim como temos índices em livros e enciclopédias, que permitem o acesso a capítulos ou páginas específicos, os esquemas relacionais também contam com índices para evitar que tenhamos de percorrer tabelas muito volumosas por inteiro, o que seria muito ineficiente.

Esse percurso completo de tabelas ou relações é conhecido como varredura sequencial (*full scan*, em inglês), e, sempre que possível, devemos evitar que ocorra na prática. Os índices são também objetos de bancos de dados relacionais que são criados como estruturas auxiliares de acesso aos dados.

Podemos definir os índices, de maneira simplificada, como estruturas contendo pares de valores referentes ao banco de dados: o primeiro valor é uma chave de busca (*search-key*), que é o valor que estamos buscando nas tabelas que compõem o esquema relacional. O segundo valor é a posição relativa (página ou bloco de disco mais registro) daquela chave de busca no arquivo que persiste os dados referentes a determinada tabela.

Na analogia com livros, a chave de busca pode ser determinado capítulo, e a posição relativa é a página em que se encontra aquele capítulo no livro. A figura a seguir ilustra essa noção de índices para o esquema das cervejas, considerando os arquivos correspondentes às tabelas Cerveja e Bar.

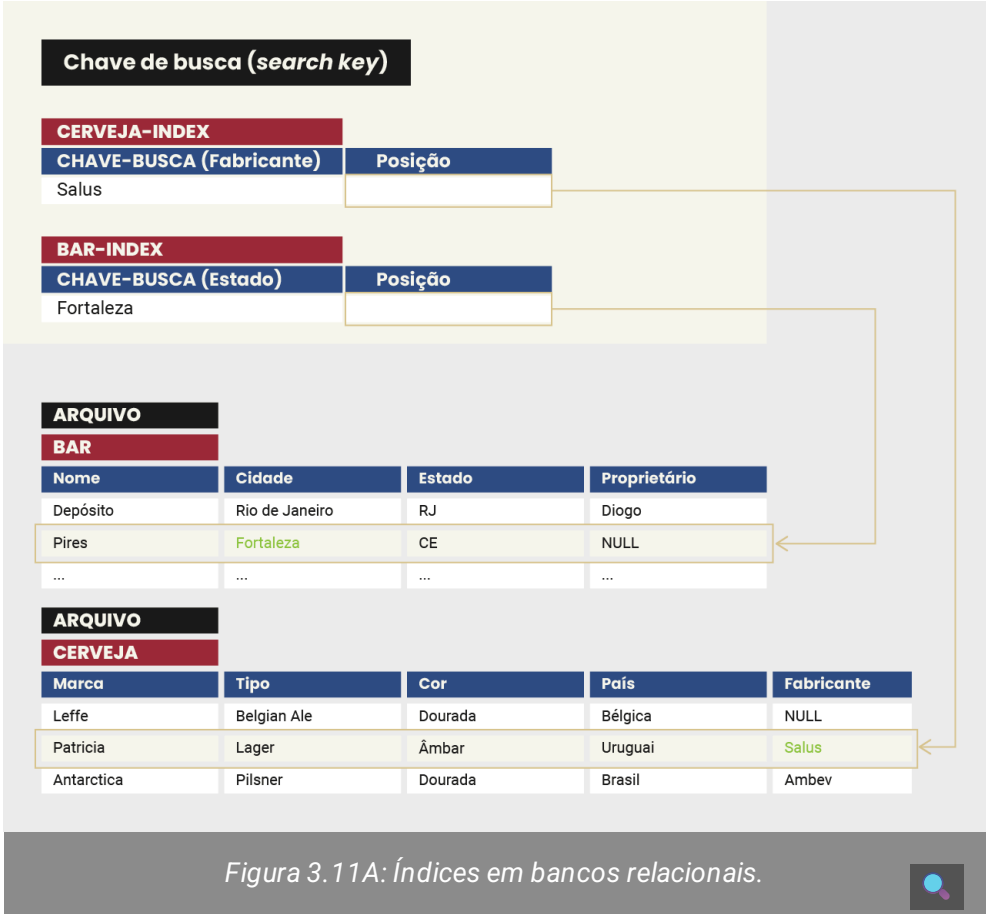


Figura 3.11A: Índices em bancos relacionais.

No caso de bancos de dados relacionais, temos, basicamente, dois tipos de índices: os índices primários e os índices secundários. Os índices são estruturas de acesso sempre ordenadas pelo campo chave de busca. Índices fazem referência aos arquivos em que os dados de tabelas são persistidos.

Para os índices primários, os arquivos referenciados pelo índice também são ordenados pela chave de busca, que, nesse caso, são únicos e não se repetem, coincidindo com a definição de chave no modelo relacional – chave candidata ou chave primária (PK). Na Figura 3.11B, temos um exemplo que ilustra de maneira simples um índice primário para o caso da tabela (e arquivo) Cerveja, em que constam os números de posição relativa de registros que implementam as tuplas de uma tabela. Não à toa índices primários remetem à ideia da existência da PK de uma tabela, que é o atributo Marca no caso da tabela Cerveja. Para criar um índice primário, basta definir uma PK com o comando **CREATE TABLE**.

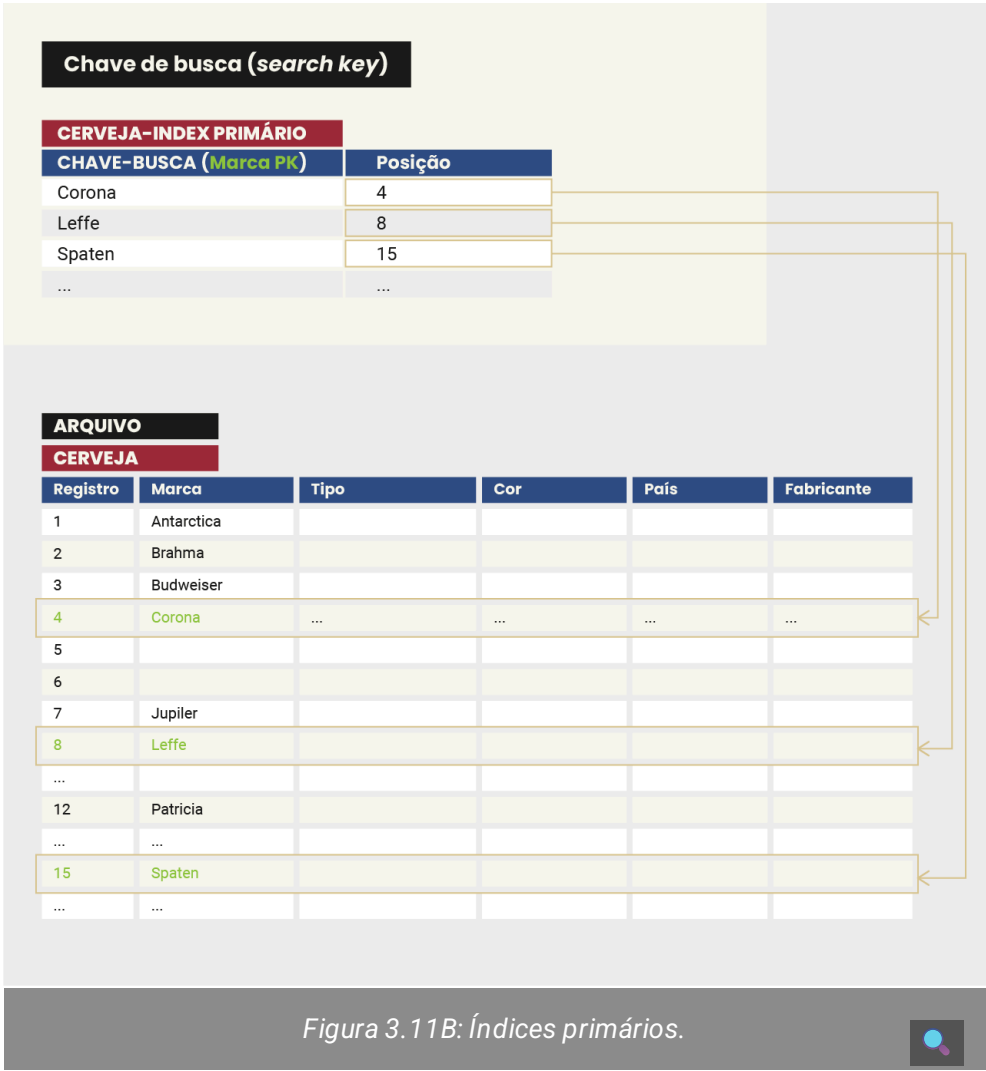
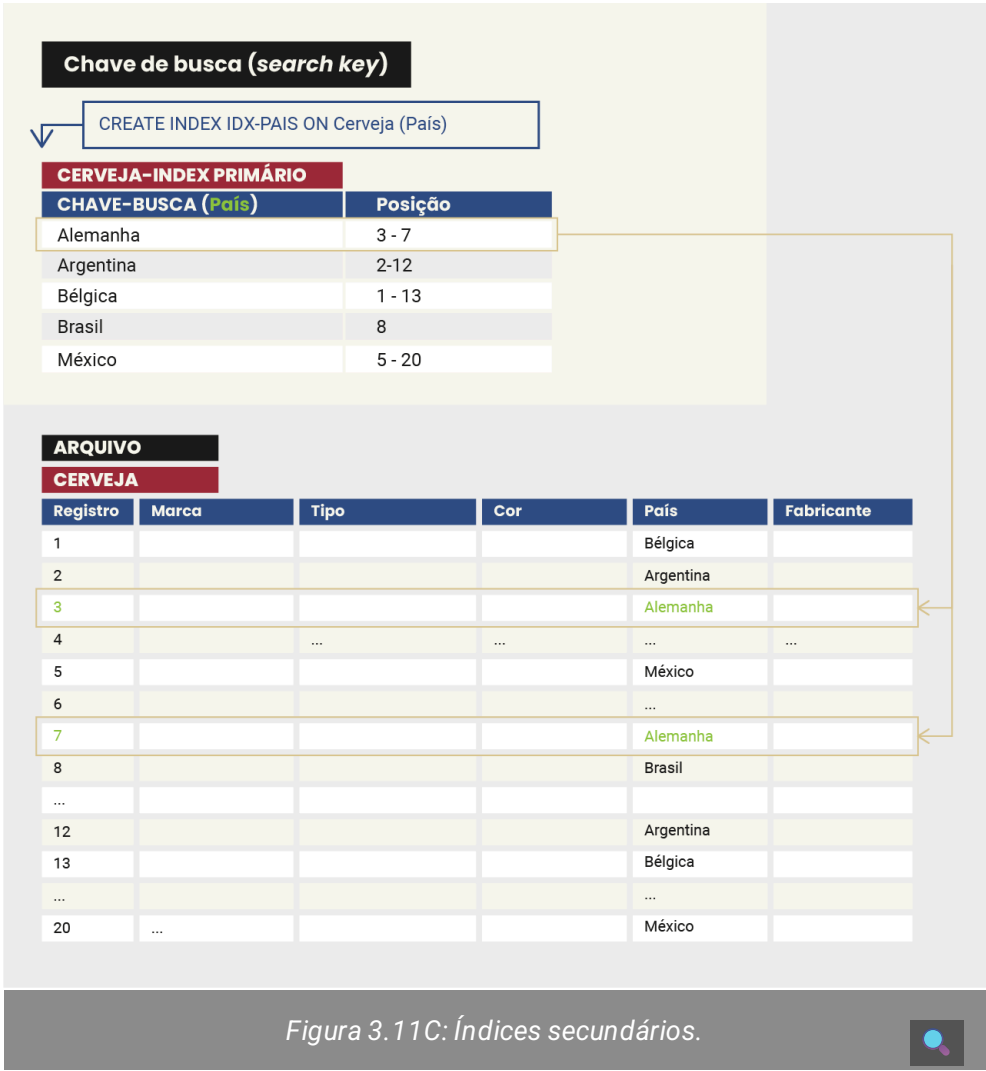


Figura 3.11B: Índices primários.

Já os índices secundários também são estruturas ordenadas que referenciam arquivos que não são ordenados pelo campo chave de busca, e podemos ter valores repetidos. A Figura 3.11C ilustra um índice secundário da mesma tabela (arquivo) com dados de cervejas, considerando agora a coluna País como referência de indexação. Enquanto índices primários são criados indiretamente na hora da criação de tabelas, os índices secundários são criados explicitamente por um comando **CREATE INDEX**. Neste exemplo, o comando respectivo poderia ser CREATE INDEX IDX-PAIS ON Cerveja (País).



Os índices, primários ou secundários, podem ser simples ou compostos. São considerados simples quando apenas uma coluna é referenciada, e compostos nos demais casos.

Existem também os chamados **índices multinível**, que são formados de estruturas de índices que referenciam outros índices. Se temos um arquivo muito grande que representa uma tabela, um índice também pode ser muito grande, precisando de **“um índice para o índice”**. As estruturas do tipo árvore B (*B-tree*) são índices multinível que são utilizados em diversos SGBDs comerciais para implementar os índices primários e secundários.

Fique ligado

Quando um sistema de banco de dados apresenta problemas de desempenho, muitas vezes o *database administrator* (DBA), ou analista responsável, opta pela criação de índices para que as consultas e transações, de maneira geral, executem em menor tempo ou em maior quantidade na mesma quantidade de tempo (vazão ou *throughput*). É esperado que o otimizador do SGBD leve em consideração tais índices, visando à melhoria do sistema como um todo.

Criar índices é uma das tarefas mais comuns que DBAs ou usuários especialistas em bancos de dados realizam na manutenção de um banco de dados. Ajustar um sistema de banco de dados que está em

produção, buscando maior eficiência, é uma atividade também conhecida por sintonia fina de banco de dados (*database tuning*).



Sabemos que no projeto conceitual e no projeto lógico de um banco de dados a ênfase é atender aos requisitos de dados e funcionais. No projeto físico, a prioridade é a *performance*, e é nesse momento que os índices são considerados, assim como as visões.

Cabe observar que uma das estratégias mais utilizadas para melhoria de desempenho em bancos relacionais é a desnormalização. Esse termo é usado para fazer referência ao processo de normalização de um banco de dados, visto na segunda aula. Desnormalizar significa, em geral, reunir duas ou mais tabelas que foram criadas separadamente em uma única tabela, evitando a execução de junções, que são operações frequentes e muito caras. Daí a importância de conhecer o processo de projeto usando formas normais, pois, caso contrário, o ato de desnormalizar um esquema relacional não faria sentido.

React de mercado

Estamos chegando ao fim de nossa aula, assista agora ao vídeo react, onde Verônica Santos, Tecnóloga em Informações Geográficas e Estatísticas, fala um pouco sobre a rotina de trabalho de DBA no IBGE.



Índices

Sabemos que podemos ter somente um índice primário para cada relação, pois este define a ordenação da tabela. Entretanto, é possível disponibilizar vários índices secundários. Os índices evitam buscas sequenciais, algo fundamental quando lidamos com grandes volumes de dados. Assim, seria natural imaginar que deveríamos criar a maior quantidade de índices possível, pois eles serão úteis para melhorar o desempenho de acesso ao banco de dados. Algo similar é feito pelos buscadores na *web*, como Google ou Bing, que, em seus sistemas de recuperação de informações (*information retrieval*), criam diversos índices para que se possa chegar rapidamente aos resultados das buscas pelas palavras-chave entradas pelos usuários. Entretanto, muitos analistas de sistemas e programadores costumam dizer que “índices são bons para fazer consultas, mas ruins para atualizações”. Você concorda com essa afirmação?

